WanderTrip – Decision-Support Travel Companion

🧭 PROJECT DEFINITION

Product Name (Working)

WanderTrip – Decision-Support Travel Companion

Core Value Proposition

Help users decide **what to do, where to go, and how to adapt**, using: - Real-time context - Reliable signals (weather, places, time, mobility, cost) - AI reasoning - Zero hallucinations / no fake data

It is a **Decision Support System**, not a booking engine.

🧠 DESIGN PRINCIPLES (NON-NEGOTIABLE)

- ✔ Decision support, not direct booking system
- ✔ Only real world signals (weather, places, mobility, timing)
- ✔ AI = reasoning layer, APIs = fact source
- ✔ Explainability always
- ✔ Fast rescheduling via context delta
- ✔ Location-agnostic (works globally)
- ✔ Legally safe integrations (no scraping, only public/legal APIs)

🏗 SYSTEM ARCHITECTURE (FINAL)

```
┌──────────────────────────────┐
│        Frontend (UI)         │
│  - Streamlit (Phase 1)       │
│  - Chat Interface            │
└──────────────────────────────┘
              │
              ↓
┌──────────────────────────────┐
│    Context Aggregation Layer │    ← REAL TIME FACTS
│  - Location & Time           │
│  - Weather                   │
│  - Places & Activities       │
│  - Mobility                  │
│  - Cost Signals              │
│  - Web References            │
└──────────────────────────────┘
              │
              ↓
┌──────────────────────────────┐
│    Decision Engine (AI)      │    ← CORE INTELLIGENCE
│  - Evaluate options          │
│  - Rank tradeoffs            │
│  - Explain Why / Why Not     │
│  - Simulate contextual change│
└──────────────────────────────┘
              │
              ↓
┌──────────────────────────────┐
│    Execution Connectors      │
│  - Maps                      │
│  - Booking Deep Links        │
│  - Transport Searches        │
└──────────────────────────────┘
```

PROJECT STRUCTURE (FINAL & IMPORTANT)

```
wandertrip/
│
├── app.py                      # Streamlit entry point
├── requirements.txt
├── .env
│
├── core/
│   ├── context/
│   │   ├── location.py         # Geo + local time + season
│   │   ├── weather.py          # Weather + forecast
│   │   ├── places.py           # Nearby places & activities
│   │   ├── mobility.py         # Travel time feasibility
│   │   ├── cost.py             # Budget awareness
│   │   ├── references.py       # Sources + verified links
│   │   └── aggregator.py       # Unified context builder
│   │
│   ├── decision/
│   │   ├── prompt.py           # Decision prompts
│   │   ├── schema.py           # Strict JSON schema
│   │   ├── engine.py           # Reasoning + ranking
│   │   └── rescheduler.py      # Context delta logic
│   │
│   ├── execution/
│   │   ├── maps.py             # Google / OSM Routes
│   │   ├── booking.py          # Deep links only
│   │   └── transport.py        # Flights / Trains / Bus
│
├── utils/
│   ├── cache.py                # TTL caching
│   ├── time.py                 # Time helpers
│   ├── validators.py           # Input validation
│   └── logger.py
│
└── data/
    └── static/
        └── country_metadata.json
```

🔍 CONTEXT AGGREGATION – DETAILED MODULES

1️⃣ location.py

Responsibilities - Reverse geocoding - Local time - Day + Season inference

APIs - Nominatim - Timezone API

Output Example

```
{
  "lat": 27.33,
  "lon": 88.60,
  "city": "Gangtok",
  "country": "India",
  "local_time": "2025-01-08T18:10",
  "day": "Wednesday",
  "season": "Winter"
}
```

2️⃣ weather.py

Responsibilities - Current weather - Short forecast - Sunset / Light availability

API - Open-Meteo

Output

```
{
  "temp_c": 11,
  "rain_probability": 0.1,
  "wind_kph": 5,
  "visibility_km": 10,
  "sunset": "17:55"
}
```

3️⃣ places.py

Responsibilities - Nearby places - Categorization - Indoor / Outdoor classification

APIs - OSM - Google Places (optional)

Output

```
{
  "places": [
    {
      "name": "MG Marg",
      "category": "shopping",
```

```
      "indoor": false,
      "distance_km": 0.4
    }
  ]
}
```


4 mobility.py

- Travel feasibility
- Walking radius
- Travel time estimation

APIs - Google Maps - OpenRouteService

5 cost.py

- Budget feasibility
- Cost tiering

Sources - Google price level - User input

6 references.py

- Provide trusted links
- No scraping
- Build credibility

Sources - Official tourism boards - Wikivoyage - Booking platforms

7 aggregator.py

- Merge context layers
- Normalize structure
- Cache optimized

🧠 DECISION ENGINE (CORE INTELLIGENCE)

prompt.py

- Defines system rules

- Safety constraints
- Reasoning instructions

schema.py

Strict JSON Output Schema

```
{
  "decision": "string",
  "why_this": ["string"],
  "why_not": ["string"],
  "alternatives": ["string"],
  "reschedule_advice": "string",
  "references": ["url"]
}
```

engine.py

- Calls Groq / Llama
- Validates JSON
- Retry logic

⚡ RESCHEDULING ENGINE

rescheduler.py - Computes context delta - Re-evaluates only impacted activities - Preserves user intent

Example Delta

```
{
  "weather_change": "rain_started",
  "time_remaining_minutes": 40
}
```

🔌 EXECUTION CONNECTORS (LEGAL SAFE – NO SCRAPING)

booking.py

- MakeMyTrip deep links
- Booking.com links

maps.py

- Google Maps navigation

💻 FRONTEND (app.py)

- Chat interface
- Context visibility optional
- Explainable decisions UI
- Quick “What If?” adjustments

⚖️ LEGAL & TRUST FRAMEWORK

✓ No scraping ✓ No fake prices ✓ Clear messaging: decision support ✓ Trusted external links ✓ Transparent source references

📅 2-MONTH DELIVERY PLAN

Weeks 1–2

- Context aggregation
- Weather + Places + Time

Weeks 3–4

- Decision Engine
- Chat UI
- Explainability layer

Weeks 5–6

- Rescheduling engine
- Multi-option comparison
- Deep linking

Weeks 7–8

- Performance & caching
- Public deployment